

MSc Önálló laboratórium 2

Téma címe: *ESP32 és STM32 alapú ipari okoskonnektor tervezése és élesztése*

Készítette: Szentgyörgyi Ábel (FN5NF0)

Külső konzulens: Karz Gergely (Artillence)

Egyetemi konzulens: Dr. Hideg Attila

Bevezetés

Az önálló laboratórium 2 tárgy keretein belül új témát kezdtem az előző féléves témám után. A téma nagyon közel áll hozzám, mivel ESP32 mikrokontrollerrel már többször foglalkoztam, illetve a BSc szakdolgozatom alapjául is ez szolgált. Az IoT, okos eszközök fejlesztése pedig kifejezetten szimpatikus volt a témában, mivel úgy látom, még sokat tud fejlődni ez az iparág és akár egyszerűbb eszközök is nagyban hozzájárulhatnak a kényelemhez, optimalizált működéshez. Azért is érdemes ilyen témákkal foglalkozni, mivel mind ipari, mind konzumer felhasználásra egyre növekvő igény van, illetve sok a kiaknázatlan terület. Több villanyszerelői képzésen bevezették a tananyag részeként az IoT eszközök telepítését is, viszont a szolgáltatói réteg még mindig nincs kellő mennyiségű szakember ezekhez a feladatokhoz. Ennek fő oka, hogy ezen eszközök telepítése egy komplexebb tudást igényel, viszont a szakemberek gyakran nem képzik magukat erre. Szükséges egy bizonyos szintű informatikai tudás is, ami tudás mérnökként az esetek nagyobb részében elérhető, így mind fejlesztéssel, mind telepítéssel érdemes lehet egy ilyen háttérrel rendelkező embernek foglalkozni.

A téma főként az ipari felhasználást veszi célul. Ahogy fejlődik a távközlés és informatika, úgy egyre nagyobb szerverparkok jönnek létre, amik elektromos kiszolgálása, működésük biztosítása és automatizált hibakezelésük egyre fontosabb szerepet kap. Az önálló labor keretein belül egy olyan okoskonnektor tervezése és fejlesztése a cél, ami ipari szervertermekben, rack szekrényekben fogja a megfelelő tápellátást biztosítani a különböző szerver PC-knek, switch-eknek, eszközöknek. Az ipari igény biztosított, hiszen az elmúlt években meghatszorozódott a datacenterekre költött pénzmennyiség, miközben maguknak a hardverkomponenseknek az ára csökkent.

Feladat leírása

Maga a feladat tehát egy okos konnektor tervezése, ami datacenterek rack szekrényeiben fog helyet kapni ipari környezetben. Ezeket a konnektorokat Power Distribution Unit-nak (PDU) nevezik a piacon. több cég is gyárt ezekhez hasonló terméket, melyik közt egyik kiemelendő a Schneider Electronic rack szintű elosztásért felelős termékei.



1. ábra Schneider Electronic rack szekrény és PDU modul [1]

Ezek főbb képességei a teljesítmény mérés, vezérelhetőség és távoli elérés. A teljesítmény mérés azért fontos, mert ha bármi hiba keletkezik a rendszerben, például leáll az egyik modul, valami hiba miatt túl nagy teljesítményfelvétel lép fel, akkor azt azonnal érzékelni lehet és szükséges beavatkozásokat lehet adott esetben automatizáltan eszközölni. A vezérelhetőség ebben az esetben főként arra vonatkozik, hogy ki és be tudjuk kapcsolni az adott aljzatot. Ez akár optimalizálási feladatot is elláthat, ha valamely eszközök használatára éppen nincs szükség, akár a kellő kapacitás megléte miatt, így könnyen ki tudjuk venni kapcsolni a rendszerből. Hibakezelésre is megoldást jelenthet ez a funkció, mivel ezzel megakadályozhatjuk a rendszer túlterhelését azonnal túláram detektálással. Távoli elérés pedig főként az előzőekben leírtak megfigyelése, távoli, egyszerű hozzáféréssel való kezelését teszi lehetővé.

További fontos kritériumai a tervezendő eszköznek, hogy legyen moduláris, hogy a könnyebb bővíthetőséget megteremtsük, láncolható, hogy az eszközök tudjanak egymásról és könnyebben el lehessen érni bármely eszközt túlzott kábelezés nélkül és egy letisztult, intuitív kezelőfelületük legyen lokálisan egy kijelzővel, hálózaton keresztül pedig egy webes felületről.

A megvalósításhoz két típusú mikrokontrollert használok, egy ESP32 és egy STM32-t. Ebben a félévben a fő cél a modulok hardveres tervezése, szoftveres implementáció elkezdése volt. A feladat megoldásához sok segítséget kaptam az Artillence Kft.-től és onnan Karz Gergelytől, aki végig segítette a munkámat.

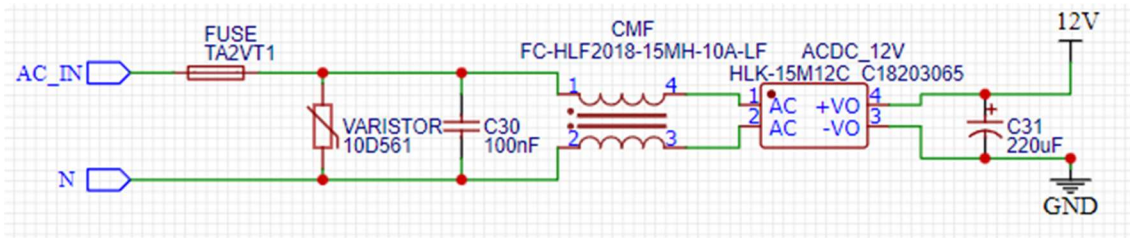
Hardveres tervezés

A hardver több modulból tevődik össze. Van egy fő modul, ami a vezérlést, kommunikációt, konfigurálhatóságot teszi lehetővé és egy PDU-ban egy darab kap helyet, ezt a továbbiakban PDU MAIN-ként azonosítom. A csatlakozó vagy aljzatok méréséhez és külön vezérléséhez minden aljzat külön modullal fog működni, amit a csatlakozó szabványának megfelelően IEC-nek fogok hívni az International Electrotechnical Commission rövidítésből (ez a szabvány foglalja magába az iparban használt csatlakozó típusokat). Végül szükség lesz egy hőmérséklet- és páratartalom mérő eszközre, amivel a környezeti tényezők is monitorozhatók lehetnek.

A tervezést az EasyEDA nevű tervező szoftverrel végeztem, aminek rendkívül egyszerű a használata, sok alkatrész footprint található benne és azonnal rendelhető a segítségével a tervezett hardver. Mivel ez volt az első ilyen jellegű tervem, ezért viszonylag több időt vett igénybe ennek teljesítése, viszont betanulás után már gyorsan tudtam módosításokat végezni.

PDU MAIN

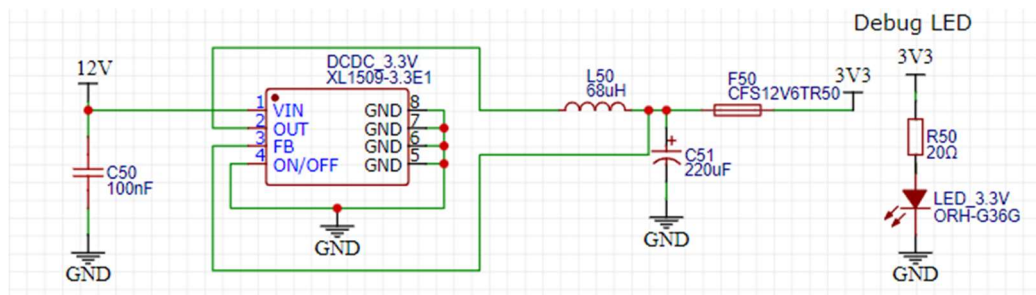
A fő modul tehát elsősorban a vezérlésért, kommunikációért felelős. A modul 3.3V-on működik, így az áramköri egységeket is ennek megfelelően kellett kiválasztani, méretezni. A fő modul táplálja a csatlakozóknál lévő IEC modulokat, az átvitel során 12V-ot alkalmaztam a nagyobb zajtűrés miatt, így ez minden modulon le van szállítva 3.3V-ra. A fő modul AC-ről van meghajtva, amit egy HI-LINK HLK-15M12C típusú AC-DC konverterrel 12V-ra alakít.



2. ábra AC-DC konverter

A schematic ábrán látható, hogy egy biztosítékot is beillesztettem, ami védelmi célt szolgál. A többi elem az AC-DC konverter adatlapi ajánlása alapján került be szintén védelmi és zavartűrési megfontolásokból.

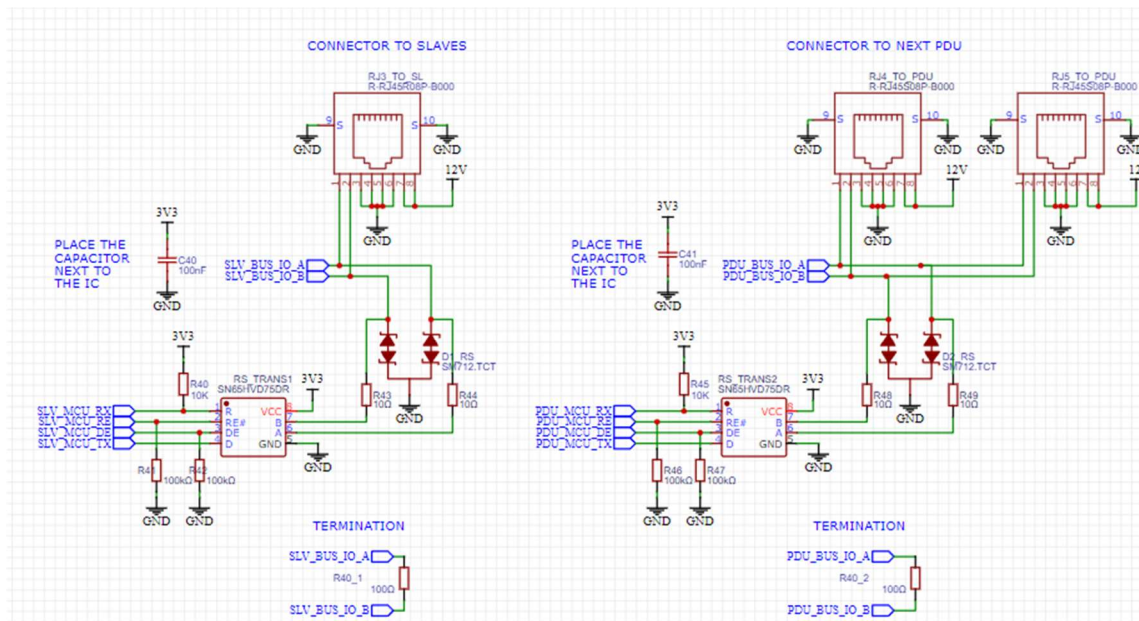
A 3.3V-ot ebből a 12V-ból egy step down DC-DC konverterrel állít elő.



3. ábra DC-DC konverter

Ennek típusa XLSEMI XL1509-3.3E1. A működésének vizsgálatához beillesztettem egy zöld színű ledet, ami 3.3V-os feszültség meglétekor fog világítani, tehát ekkor feszültség alatt van a rendszer.

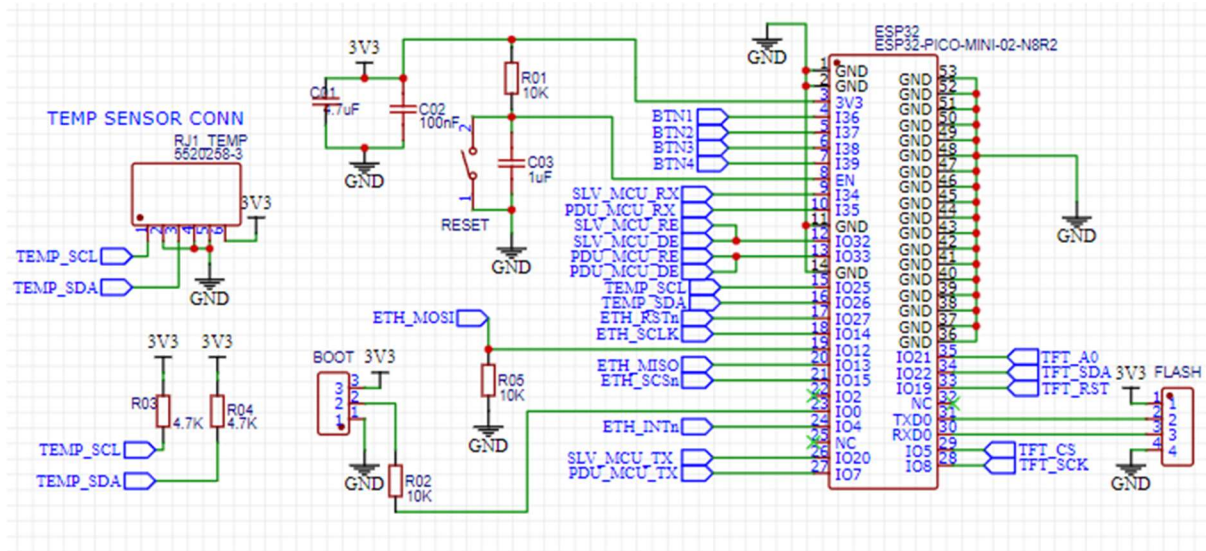
A kommunikáció RS485-ös [2] elektronikai szabványt használja. Az RS485 egy szimmetrikus átviteli mód, ami azt jelenti, hogy két (A és B) kimeneteken érkező jeleket biztosít, amit egy buszról lehet elérni. Multidrop megvalósítása van, így a buszon lévő összes eszköz minden üzenetet megkap, az, hogy melyik szól neki, már a protokoll megvalósításán múlik. Ennek megfelelően szükség van egy master eszközre, ami kezeli a buszt, így ő kommunikál a slave-ekkel. Alap esetben 32 eszköz lehet egy buszon, ami így 32 unit terhelést (Unit Load = UL) jelent. A vevő egységek méretezésével ez a 32 eszköz bővíthető akár 255 eszközre is, azzal, hogy a transceiver-ek unit terhelését csökkentjük. Ebben az esetben a SN65HVD75 típusú transceiver 3/20 UL terhelést tesz a buszra egységenként, így legfeljebb 213 csatlakozhat a buszhoz [3]. Számunkra ez megfelelően elég, mivel várhatóan legfeljebb 20-30 eszköz fog egy buszon lenni.



A fő modulba 2 db transceiver szükséges, mivel a PDU MAIN más PDU MAIN-ekkel is össze lesz kötve és egy PDU-n belül az IEC modulok is vele kommunikálnak majd egy másik buszon. A csatlakozás RJ45-ös csatlakozókkal valósul meg UTP kábel pedig a sodort érpárjával szállítja az A és B csatorna jeleit, illetve a föld és 12V is átmegy rajta.

A távoli elérést egy W5500-as ethernet modul valósítja meg. Ez SPI interfacen kommunikál majd a fő modul ESP32-jével és szintén egy RJ45-ös csatlakozón keresztül kapcsolódik a hálózathoz.

Maga a mikrokontroller egy ESP32-PICO-MINI-02-N8R2 típusú modul. Kiválasztásakor azt vettük figyelembe, hogy minél több GPIO lába legyen, amivel a perifériák csatlakoztathatók rá.



6. ábra ESP32

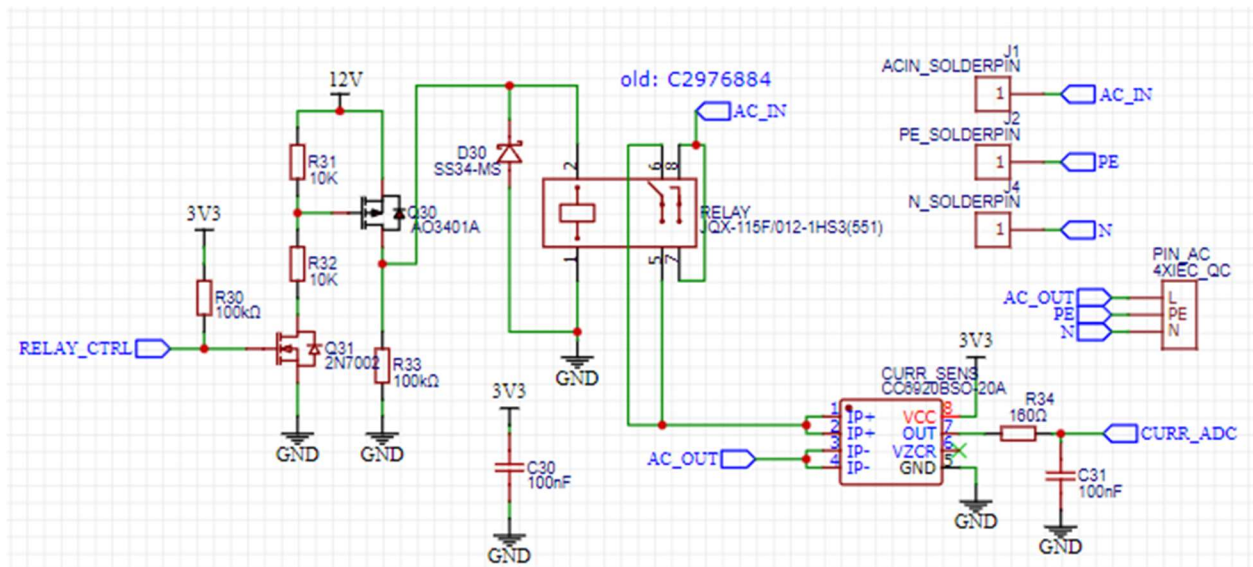
Amire különösen figyelni kellett, hogy SPI boot mode-ban és download boot mode-ban (amikor betölti a szoftverét és amikor flasheljük kívülről) más más strapping pin-eknek kell megfelelő állapotban lenniük [4]. A legfontosabbak az IO0, IO2 láb. Az IO12 lábunk LOW állapotban kell lennie bootoláskor, mivel ez alapján dől el, hogy 3.3V üzemel az eszköz. Ide bekerült még egy reset gomb, amivel megkönnyíthetjük a módok közti váltást (IO0 HIGH vagy LOW állapotban bekapcsoláskor) újraindítás esetén. A külső flashelése a TXD0 és RXD0 lábak segítségével UART-on keresztül fog megvalósulni, így ezeket a lábakat kivezettem.

Került még a PDU MAIN-re egy Newvisio N177-1216TCWPG01-H14 típusú 1.8" méretű TFT kijelző, ami szinten SPI-lal kommunikál, illetve 4 nyomógomb.

IEC modul

Az aljazoknál lévő modul fő feladata az árammérés és ki- be kapcsolás vezérlése. Ezen áramkör kommunikációs és táp megoldásai megegyeznek az előbbieken említett PDU MAIN részeivel, így azokat itt nem részletezném.

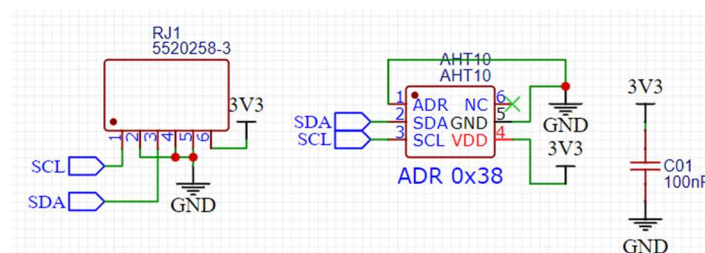
Az árammérést egy Hall szenzoros árammérővel oldottam meg. A megoldás még hiányos, mivel csak árammérésre van felkészítve az áramkör, viszont teljesítmény pontos méréséhez szükség lesz a feszültség mérésére is. Az árammérő kimenetén van még egy 10kHz vágási frekvenciájú aluláteresztő szűrő, hogy a zajokat csökkentsük. A vezérléshez egy normally open üzemeltetésű 16A-es relay került be.



7. ábra HALL szenzoros árammérő és relay

AHT10-es hőmérséklet- és paratartalom mérő

Mivel fontos monitorozni a szerverterem környezeti viszonyait is, készült egy hőmérséklet mérésre alkalmas áramkör is egy AHT10-es szenzorral. Ennek felépítése meglehetősen egyszerű. A szenzor I2C-n kommunikál, illetve egy RJ11-es csatlakozóval lehet majd a PDU MAIN áramkörhöz csatlakoztatni, ahol az ESP32 fogja a mérési adatokat kapni.



8. ábra AHT10 áramköre

Szoftver

Mivel első körben a hardver került a fő fókuszba, a szoftveres megvalósításra limitáltabb idő jutott. A kommunikáció megvalósítása tűnt az egyik legfontosabb feladatnak, így főleg azzal foglalkoztam szoftveres szinten. Az ESP32 fejlesztését Arduino IDE-ban, az STM32-höz pedig STMCubeIDE-t használtam.

Modbus

A kommunikációhoz RS485 felett Modbus protokollt használok. A Modbus egy széles körben használt, egyszerű és megbízható kommunikációs protokoll, amelyet elsősorban ipari automatizálási rendszerekben alkalmaznak. Az 1979-ben a Modicon (ma Schneider Electric) [5] által kifejlesztett protokoll célja az eszközök közötti adatsere megkönnyítése, függetlenül azok gyártójától. A Modbus egy master-slave (vagy újabb nevén client-server) architektúrán alapul, ahol egy master kezdeményezi a kommunikációt, és egy slave válaszol. A protokoll alapvetően két fő változata létezik:

- **Modbus RTU** (Remote Terminal Unit): Bináris formátumú adatkereteket használ, amely hatékony és kis méretű üzenetváltást biztosít.
- **Modbus ASCII**: Olvashatóbb, karakter alapú adatkereteket alkalmaz, de kevésbé hatékony az RTU-hoz képest.

Ezek közül én a Modbus RTU-t használom. Egy Modbus RTU üzenet az alábbi részekből áll:

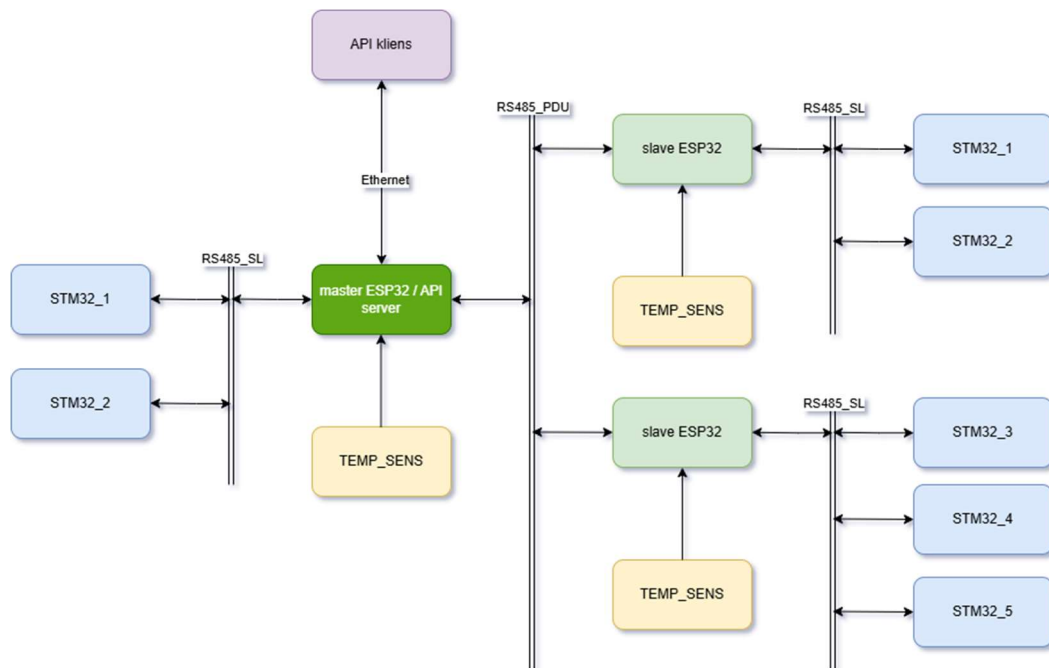
- **Címmező** (1 bájt): Meghatározza a slave eszköz címét.
- **Funkciókód** (1 bájt): Leírja, hogy milyen műveletet kér a master (pl. olvasás, írás).
- **Adatmező** (n bájt): A művelethez kapcsolódó adatok.
- **CRC** (2 bájt): Hibadetektálás céljából.

Modbus RTU működése úgy néz ki, hogy a slave-nek van 4 különböző memória tároló egysége: Holding Register (több byte), Coil (1 bit), Input Register (több byte), Discrete Inputs (1 bit). A Holding Register és Coil olyan tárolók, amikbe a master írási műveletet is kezdeményezhet, Input Registerbe és Discrete Input-ba viszont csak a slave írhat, kívülről csak olvasni lehet. A master minden üzenetben leírja, hogy pontosan milyen műveletet szeretne végezni és melyik tároló mely részét szeretné elérni. Ez ehhez a feladathoz kifejezetten jó módszer, hiszen például a Coil tárolhatja a relay állapotát, amit

a master írni is tud, és a teljesítmény érték lehet a Input Registerben, amit kívülről csak olvasni tud a master.

Azért esett a Modbus-ra a választásom, mivel egy iparban is gyakran használt protokollról van szó, tehát akár más eszközökkel való későbbi kompatibilitás is megoldott lehet vele, illetve léteznek hozzá megírt könyvtárak is, amiket könnyen lehet használni, mint Arduino IDE környezetben, mind STMCubeIDE-val. További haszna, hogy rendesen dokumentált protokollról van szó, definiált hibaüzenetekkel, hibakódokkal, amik tovább egyszerűsítik a vevők számára a használatot, esetleg a debuggolást fejlesztői oldalról.

A kommunikáció PDU MAIN – PDU MAIN és PDU MAIN – IEC áramkörök között 2 féle buszon történik ahogy a 9. ábrán ez látható is:



9. ábra Kommunikációs buszok

Az RS485_SL buszok magában a PDU-ban vannak, ennek masterje minden esetben az egyetlen PDU MAIN, ami az adott PDU-ban található. Az RS485_PDU busz különböző PDU-k láncolásából jön. Itt kritikus pont, hogy melyik PDU MAIN lesz a master, mivel egy buszon csak egy master lehet egyszerre. Ennek ideiglenes megoldása az, hogy az a PDU lesz a master, amihez Etherneten hálózatra van kötve. Ekkor szoftver szinten ez a PDU MAIN ESP32-je masterként inicializálódik bekapcsoláskor, a többi PDU MAIN pedig slave-ként. Ezzel a probléma, hogy elvárjuk, hogy mindenképp legfeljebb ESP32 legyen ethernetes hálózaton, ha több van, akkor már dönteni kell közülük. Itt jön képbe

a Modbus protokoll, amit az RS485-ön felül használok. A protokoll egy egyszerű master-slave kommunikációt valósít meg, viszont előre tudni kell, hogy melyik eszköz a master, a slaveknek pedig egy cím ID-ra van szükségük, amiket ismernie kell a masternek, hogy tudjon velük kommunikálni. Az előzőkből adódóan a probléma egyrészt az, hogy nem tudjuk pontosan bekapcsolás előtt, hogy kinek van ethernetes kapcsolata, így nem tudjuk eldönteni előre a master kilétét. Második probléma, hogy nem ismeri a címeket a master, így nem tudja mivel kommunikáljon.

A problémákra megoldást jelenthet, hogyha készítünk egy custom protokollt, ami Modbus-tól függetlenül képes begyűjteni az összes infót (van-e ethernetes kapcsolat és kinek), ID-kat, és miután ez lefutott, el lehet dönteni, ki legyen a Modbus master. Ennél további nehézséget jelent, hogy szükség van arra is, hogy minden eszköz eltérő időközönként használja a buszt. Ezt esetleg egy olyan default késleltetés megoldhatja, ami minden eszköznél eltérő idő késleltetése után használja a buszt, viszont ezzel sem lehet kizárni az ütközést, így szükségesek további lépések.

Egy másik megoldás lehet, hogy manuálisan konfiguráljuk a hálózatot egy általunk kiválasztott masteren, így biztosak lehetünk a címekben és kizárható az ütközés, viszont ezzel elvesztjük az automatikus beállítását a rendszernek.

Eredmények, fejlesztési lehetőségek

A félév során a hardveres tervek lettek elkészítve teljesen, amikhez a PCB-k is megrendelésre kerültek. Élesztésükig még nem jutottunk el, de még idén arra is sor kerül. A hardverrel kapcsolatban folyamatosan jöttek fel fejlesztési lehetőségek:

- Erősebb Relay: A jelenlegi 16A-est ki lehetne cserélni egy nagyobb, 32A-esre.
- Teljesítmény mérése: Feszültség méréssel ki kell egészíteni a mérési módszert, hogy pontosabb eredményt kapjunk és a feszültség ingadozásokból is lehessen hibajelenségekre következtetni.
- Hardver design:
 - Minden alkatrészt egy oldalra kell rakni, hogy olcsóbb legyen a gyártás
 - Az ESP32 esetén olyan modult választottunk, amihez speciális beültetési eljárás kellett, ami szintén magasabb költséget jelentett, ehelyett találni kell egy másikat, amihez nem szükséges ez a forrasztási mód.

- Csökkenteni kell a használt GPIO lábakat az ESP32-n, például a kijelző lecserélésével egy I2C kommunikációsra SPI helyett.
- Le kell csökkenteni a PDU MAIN nyák hosszát, hogy optimálisabban elférjen a tervezett házban.
- Külön nyákon jobb lenne a kijelző, mivel azzal is helyet pótolnánk, illetve könnyebben beilleszthető lenne vele a házba.

Szoftveres oldalon több hiányzó rész is van, amiket pótolni kell a jövőben. A kommunikáció működik, viszont valami miatt CRC hiba jelentkezik. Ennek javítása érdekében jobban bele kell menni a Modbus könyvtárainak esetleges hibáiba, mivel külön könyvtár kell az ESP-hez és STM-hez. Ezen felül a korábban taglalt master döntést is implementálni kell, amihez valószínűleg egy saját custom protocooll lesz a célszerű.

A hiányzó szoftver részek közül néhány már alap szinten működik. A kijelzőre ki tudunk írni egyszerű szövegeket, a távoli eléréshez szükséges web server az ESP32-n fut és elérhető kívülről, viszont még nagyon kezdetleges. Az STM32 kódja szintén alakulóban van, viszont még szükség van nagyobb hibakezelésre, az adatok mentésére.

Összegzés

Összességében úgy gondolom, hogy sikerült egy jó szintre eljutni egy félév alatt ezzel a témával, és ezek után nagyobb hangsúlyt kaphat a szoftveres implementáció. Sokat tanultam a projectből a félév során, kipróbáltam a hardver design folyamatát, sok alkatrész működését megismertem és gyakoroltam a nyák tervezését is. Szoftveres oldalról izgalmas feladatok állnak még előttem, főként a kommunikáció, PDU MAIN master – slave döntés automatizálása, és a hiányzó/hiányos részek implementálása kapcsán. Tanulásgként levonhatom, hogy a hardveres tervezéshez mindig egy átgondolt rendszerszemlélettel kell nekiállni, a szoftveres implementációt pedig inkább részenként tökéletesítem, mert úgy felgyorsítom és megkönnyítem a debuggolást is. A témát szeretném továbbvinni diplomatervnek is.

Ezúton köszönöm még egyszer Karz Gergely segítségét, illetve az általa és az Artillence Kft.-től kapott eszközöket, PCB rendelést.

Hivatkozások

- [1] S. Electronic. [Online]. Available: <https://www.se.com/>.
- [2] RS485. [Online]. Available: <https://hu.wikipedia.org/wiki/RS%E2%80%93485>.
- [3] R. transceiver. [Online]. Available:
https://www.lcsc.com/datasheet/lcsc_datasheet_1810181621_Texas-Instruments-SN65HVD75DR_C57928.pdf.
- [4] „ESP32,” [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp32-pico-mini-02_datasheet_en.pdf.
- [5] modbus. [Online]. Available: <https://en.wikipedia.org/wiki/Modbus>.